```
#Group: Manan Patel & Dev Amin

code3rdday.pck <-
c("code3rdday.pck", "codeday1.pck", "my.dat.plot0", "gui.datplot",
"NOAA", "codeday3.pck", "my.dat.plot3", "my.dat.plot4", "codeday4.pck",
"my.bootstrap1", "gui.bootstrap1", "my.bootstrap2", "gui.bootstrap2",
"my.bootstrap3", "gui.bootstrapxy", "my.dat.plot5", "my.dat.plot5a"
)
codeday1.pck <-
c("codeday1.pck", "my.dat.plot0", "gui.datplot", "NOAA")

# my.dat.plot0: plots two chosen NOAA columns with a smoothing spline
my.dat.plot0 <-
function(mat=NOAA, i=3, j=2, zxlab, zylab, zmain, zcol) {
  # OLD EXAMPLES (commented by prof):
  # plot(NOAA[,3],NOAA[,2])                     # raw scatter
  # lines(smooth(NOAA[,3],NOAA[,2]))            # attempt with smooth()
  # traceback()                                 # shows error history if it failed
  # lines(smooth.spline(NOAA[,3],NOAA[,2]))     # correct smoother {cubic spline)

  # ---- Acctual working code ----
  plot(mat[,i], mat[,j],
       xlab = zxlab,       #label for x-axis
       ylab = zylab,       #label for y-axis
       main = zmain)       # plot title

  # add a smoothing spline fit of y ~ x
  # (lecture: spline smooths out curvature, has effective df)
  lines(smooth.spline(mat[,i], mat[,j]),
        col = zcol)        # spline line drawn inchosen color
}

gui.datplot <-
function(){
library(tcltk)
inputs <- function(){

   x <- tclVar("NOAA")
   y <- tclVar("1")
   z <- tclVar("2")
   w<-tclVar("\"delta temp\"")
wa<-tclVar("\"disasters\"")
wb<-tclVar("\"Disasters vs warming\"")
zc<-tclVar("2")

   tt <- tktoplevel()
   tkwm.title(tt,"Choose parameters for new function                ")
   x.entry <- tkentry(tt, textvariable=x)
   y.entry <- tkentry(tt, textvariable=y)
   z.entry <- tkentry(tt, textvariable=z)
   w.entry<-tkentry(tt, textvariable=w)
wa.entry<-tkentry(tt,textvariable=wa)
wb.entry<-tkentry(tt,textvariable=wb)
zc.entry<-tkentry(tt,textvariable=zc)
   reset <- function()
    {
     tclvalue(x)<-""
     tclvalue(y)<-""
     tclvalue(z)<-""
        tclvalue(w)<-""
tclvalue(wa.entry)<-""
tclvalue(wb.entry)<-""
tclvalue(zc.entry)<-""
       }

   reset.but <- tkbutton(tt, text="Reset", command=reset)

   submit <- function() {
     x <- tclvalue(x)
     y <- tclvalue(y)
     z <- tclvalue(z)
       w<-tclvalue(w)
        wa<-tclvalue(wa)
        wb<-tclvalue(wb)
        zc<-tclvalue(zc)
     e <- parent.env(environment())
     e$x <- x
     e$y <- y
     e$z <- z
e$w<-w
e$wa<-wa
e$wb<-wb
e$zc<-zc
        tkdestroy(tt)
```

```
   }

   submit.but <- tkbutton(tt, text="start", command=submit)
   tkgrid(tklabel(tt,text="Input data matrix"),columnspan=2)
   tkgrid(tklabel(tt,text="data"), x.entry, pady=10, padx =30)


   tkgrid(tklabel(tt,text="Input X column"),columnspan=2)
   tkgrid(tklabel(tt,text="i"), y.entry, pady=10, padx =30)


   tkgrid(tklabel(tt,text="Input Y column"),columnspan=2)
   tkgrid(tklabel(tt,text="j"), z.entry, pady=10, padx =30)

  tkgrid(tklabel(tt,text="Input xaxis label"),columnspan=2)
   tkgrid(tklabel(tt,text="Xaxis"), w.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input yaxis label"),columnspan=2)
   tkgrid(tklabel(tt,text="Yaxis"), wa.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input plot title"),columnspan=2)
   tkgrid(tklabel(tt,text="Plot title"), wb.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input color code"),columnspan=2)
   tkgrid(tklabel(tt,text="Color code"),zc.entry, pady=10, padx =30)

   tkgrid(submit.but, reset.but)

  tkwait.window(tt)
  return(c(x,y,z,w,wa,wb,zc))
}
#now run the function like:
predictor_para <- inputs()
print(predictor_para)
mat<-eval(parse(text=predictor_para[1]))
ind1<-eval(parse(text=predictor_para[2]))
ind2<-eval(parse(text=predictor_para[3]))
xlab<-eval(parse(text=predictor_para[4]))
ylab<-eval(parse(text=predictor_para[5]))
maintitle<-eval(parse(text=predictor_para[6]))
zcol<-eval(parse(text=predictor_para[7]))
my.dat.plot0(mat,ind1,ind2,xlab,ylab, maintitle,zcol)

}
NOAA <-
structure(list(year = 1980:2016, X.disaster = c(3L, 1L, 3L, 5L,
2L, 5L, 2L, 0L, 1L, 5L, 3L, 4L, 6L, 4L, 6L, 4L, 4L, 3L, 9L, 5L,
2L, 2L, 4L, 7L, 5L, 5L, 6L, 5L, 11L, 7L, 5L, 16L, 11L, 9L, 8L,
10L, 15L), delta.temp = c(0.27000000000000002, 0.33000000000000002,
0.13, 0.29999999999999999, 0.16, 0.12, 0.19, 0.33000000000000002,
0.40999999999999998, 0.28999999999999998, 0.44, 0.42999999999999999,
0.23000000000000001, 0.23999999999999999, 0.32000000000000001,
0.45000000000000001, 0.34999999999999998, 0.47999999999999998,
0.63, 0.41999999999999998, 0.41999999999999998, 0.54000000000000004,
0.63, 0.62, 0.55000000000000004, 0.68999999999999995, 0.64000000000000001,
0.66000000000000003, 0.54000000000000004, 0.65000000000000002,
0.71999999999999997, 0.60999999999999999, 0.64000000000000001,
0.66000000000000003, 0.75, 0.87, 0.98999999999999999)), class = "data.frame", row.names = c(NA,
-37L))
codeday3.pck <-
c("codeday3.pck", "my.dat.plot3", "my.dat.plot4")

my.dat.plot3 <-
  function(mat=NOAA,i=3,j=2,zxlab,zylab,zmain,zcol,do.sqrt=F){
    #if do.sqrt=FALSE: plot raw y-values
    #If do.sqrt=TRUE: apply sqrt transform (variance stabilization for Poisson counts)
    if(!do.sqrt){
      plot(mat[,i], mat[,j], xlab=zxlab, ylab=zylab, main=zmain)
      # flexible smoothing spline (df chosen by cross-validation)
      smstr <- smooth.spline(mat[,i], mat[,j])
      lines(smstr, col=zcol)
    } else {
      plot(mat[,i], sqrt(mat[,j]), xlab=zxlab, ylab=zylab, main=zmain)
      smstr <- smooth.spline(mat[,i], sqrt(mat[,j]))
      lines(smstr, col=zcol)
    }

    # Compute residuals => observed y - predicted y from spline
    # residuals are used later in F-test andbootstrap
    resid <- mat[,j] - predict(smstr, mat[,i])$y
    smstr$resid <- resid

    # Return smoothing spline object (with residuals attached )
    smstr
```

```
  }


my.dat.plot4 <-
  function(mat=NOAA,i=3,j=2,zxlab,zylab,zmain,zcol,do.sqrt=F){
    #simmilar to my.dat.plot3 but compares 2 fits:
    # (1) flexible spline (df chosen by CV)
    # (2) linear-ish spline forced to df=2
    if(!do.sqrt){
      plot(mat[,i], mat[,j], xlab=zxlab, ylab=zylab, main=zmain)
      smstr     <- smooth.spline(mat[,i], mat[,j])
      smstr.lin <- smooth.spline(mat[,i], mat[,j], df=2)   # ~line
      lines(smstr,     col=zcol)       # flexible fit
      lines(smstr.lin, col=zcol+1)     # linear fit
    } else {
      plot(mat[,i], sqrt(mat[,j]), xlab=zxlab, ylab=zylab, main=zmain)
      smstr     <- smooth.spline(mat[,i], sqrt(mat[,j]))
      smstr.lin <- smooth.spline(mat[,i], sqrt(mat[,j]), df=2)
      lines(smstr,     col=zcol)
      lines(smstr.lin, col=zcol+1)
    }
  }

#resid1<-mat[,j]-predict(smstr,mat[,i])$y
#resid2<-mat[,j]-predict(smstr.lin,mat[,i])$y
#dfmod<-smstr$df
#dflin<-2
#ssmod<-sum(resid1^2)
#sslin<-sum(resid2^2)
#numss<-sslin-ssmod
#n1<-length(mat[,j])
#Fstat<-(numss/(dfmod-dflin))/(ssmod/(n1-dfmod))
#pvalue<-pf(Fstat,dfmod-dflin,n1-dfmod)
#stat.out<-list(SSM=ssmod,SSLIN=sslin,dfmod=dfmod,dflin=dflin,F=Fstat,P=pvalue,n=n1)
#smstr$statcalc<-stat.out
#smstr

codeday4.pck <-
c("codeday4.pck", "my.bootstrap1", "gui.bootstrap1", "my.bootstrap2",
"gui.bootstrap2", "my.bootstrap3", "gui.bootstrapxy", "my.dat.plot5",
"my.dat.plot5a")

# my.bootstrap1: residual bootstrap to test curvature (flexible spline vs df=2 line)
my.bootstrap1 <-
  function(mat=NOAA, i=3, j=2, zxlab, zylab, zmain, zcol,
           do.sqrt=F, nboot=10000){

    #  setup: 2 side-by-side panels (original F + bootstrap dist)
    par(mfrow=c(1,2))

    # Geting baseline fit + stats using my.dat.plot5 (flexible vs linear spline)
    # do.plot=TRUE (show curves), in.boot=FALSE (normal run)
    stat.out0 <- my.dat.plot5(mat, i, j, zxlab, zylab, zmain,
                              zcol, do.sqrt, do.plot=T, in.boot=F)

    #observed F-statistic comparing spline vs line
    F0 <- stat.out0$F

    # Residuals from the linear fit (null model)
    resid0 <- stat.out0$resid.lin

    # empty vector to store bootstrap F-statistics
    bootvec <- NULL

    #Fitted values from linear spline(null fit)
    y0 <- predict(stat.out0$smstrlin, mat[,i])$y

    # Copy of matrix for bootstrap samples
    matb <- mat

    # Main bootstrap loop
    for(i1 in 1:nboot){
      # progress message every 500 iterations
      if(floor(i1/500) == (i1/500)){ print(i1) }

      # Resample residuals with replacement
      residb <- sample(resid0, replace=T)

      # Generate new y-values under null = fitted line + resampled residuals
      Yb <- y0 + residb
      matb[,j] <- Yb

      #refit flexible vs linear spline to bootstrap sample
      stat.outb <- my.dat.plot5(matb, i, j, zxlab, zylab, zmain,
```

```
                                zcol, do.sqrt, do.plot=F, in.boot=T)

      # save bootstrap F-statistic
      bootvec <- c(bootvec, stat.outb$F)
    }

    # bootstrap p-value = fraction of bootstrap F's more extreme than observedF0
    pvalboot <- sum(bootvec > F0) / nboot

    # Show bootstrap distribution
    boxplot(bootvec)

    #attach p-value to output list
    stat.out0$pvalboot <- pvalboot

    # Return results
    stat.out0
  }

gui.bootstrap1 <-
function(){
library(tcltk)
inputs <- function(){

   x <- tclVar("NOAA")
   y <- tclVar("1")
   z <- tclVar("2")
   w<-tclVar("\"delta temp\"")
wa<-tclVar("\"disasters\"")
wb<-tclVar("\"Disasters vs warming\"")
zc<-tclVar("2")
qc<-tclVar("F")
rc<-tclVar("10000")

   tt <- tktoplevel()
   tkwm.title(tt,"Choose parameters for new function")
   x.entry <- tkentry(tt, textvariable=x)
   y.entry <- tkentry(tt, textvariable=y)
   z.entry <- tkentry(tt, textvariable=z)
   w.entry<-tkentry(tt, textvariable=w)
wa.entry<-tkentry(tt,textvariable=wa)
wb.entry<-tkentry(tt,textvariable=wb)
zc.entry<-tkentry(tt,textvariable=zc)
qc.entry<-tkentry(tt,textvariable=qc)
rc.entry<-tkentry(tt,textvariable=rc)
   reset <- function()
    {
     tclvalue(x)<-""
     tclvalue(y)<-""
     tclvalue(z)<-""
        tclvalue(w)<-""
tclvalue(wa.entry)<-""
tclvalue(wb.entry)<-""
tclvalue(zc.entry)<-""
tclvalue(qc.entry)<-""
tclvalue(rc.entry)<-""
       }

   reset.but <- tkbutton(tt, text="Reset", command=reset)

   submit <- function() {
     x <- tclvalue(x)
     y <- tclvalue(y)
     z <- tclvalue(z)
       w<-tclvalue(w)
        wa<-tclvalue(wa)
        wb<-tclvalue(wb)
        zc<-tclvalue(zc)
        qc<-tclvalue(qc)
        rc<-tclvalue(rc)
     e <- parent.env(environment())
     e$x <- x
     e$y <- y
     e$z <- z
e$w<-w
e$wa<-wa
e$wb<-wb
e$zc<-zc
e$qc<-qc
e$rc<-rc
        tkdestroy(tt)
   }

   submit.but <- tkbutton(tt, text="start", command=submit)
```

```
   tkgrid(tklabel(tt,text="Input data matrix"),columnspan=2)
   tkgrid(tklabel(tt,text="data"), x.entry, pady=10, padx =30)


   tkgrid(tklabel(tt,text="Input X column"),columnspan=2)
   tkgrid(tklabel(tt,text="i"), y.entry, pady=10, padx =30)


   tkgrid(tklabel(tt,text="Input Y column"),columnspan=2)
   tkgrid(tklabel(tt,text="j"), z.entry, pady=10, padx =30)

  tkgrid(tklabel(tt,text="Input xaxis label"),columnspan=2)
   tkgrid(tklabel(tt,text="Xaxis"), w.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input yaxis label"),columnspan=2)
   tkgrid(tklabel(tt,text="Yaxis"), wa.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input plot title"),columnspan=2)
   tkgrid(tklabel(tt,text="Plot title"), wb.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input color code"),columnspan=2)
   tkgrid(tklabel(tt,text="Color code"),zc.entry, pady=10, padx =30)

tkgrid(tklabel(tt,text="Input sqrt indicator"),columnspan=2)
   tkgrid(tklabel(tt,text="Sqrt=T"),qc.entry, pady=10, padx =30)

tkgrid(tklabel(tt,text="Input nboot"),columnspan=2)
   tkgrid(tklabel(tt,text="nboot"),rc.entry, pady=10, padx =30)

   tkgrid(submit.but, reset.but)

  tkwait.window(tt)
  return(c(x,y,z,w,wa,wb,zc,qc,rc))
}
#Now run the function like:
predictor_para <- inputs()
print(predictor_para)
mat<-eval(parse(text=predictor_para[1]))
ind1<-eval(parse(text=predictor_para[2]))
ind2<-eval(parse(text=predictor_para[3]))
xlab<-eval(parse(text=predictor_para[4]))
ylab<-eval(parse(text=predictor_para[5]))
maintitle<-eval(parse(text=predictor_para[6]))
zcol<-eval(parse(text=predictor_para[7]))
zsqrt<-eval(parse(text=predictor_para[8]))
znboot<-eval(parse(text=predictor_para[9]))
my.bootstrap1(mat,ind1,ind2,xlab,ylab, maintitle,zcol,zsqrt,znboot)

}

# my.bootstrap2: residual bootstrap for confidence/prediction bands
# - uses flexible spline (smstrmod) and refits with resampled residuals
# - can build either prediction or confidence bounds
# - Can use percentile or pivotal intervals
my.bootstrap2 <-
  function(mat=NOAA, i=3, j=2, zxlab, zylab, zmain, zcol,
           do.sqrt=F, nboot=10000,
           pred.bound=T,     # if TRUE → prediction band; else CI for mean
           conf.lev=.95,     # confidence level (default 95%)
           pivotal=T){       # if TRUE → use pivotal; else percentile

    par(mfrow=c(1,1))

    # Baseline spline fit (flexible vs linear) and stats
    stat.out0 <- my.dat.plot5(mat, i, j, zxlab, zylab, zmain,
                              zcol, do.sqrt, do.plot=T, in.boot=F)

    #Residuals from flexible fit (model-based)
    resid0 <- stat.out0$resid.mod

    #Storage for bootstrap fits
    bootmat <- NULL

    # fitted values from flexible spline
    y0 <- predict(stat.out0$smstrmod, mat[,i])$y
    matb <- mat

    for(i1 in 1:nboot){
      if(floor(i1/500) == (i1/500)){ print(i1) }

      # resample residuals (classic residual bootstrap)
      residb <- sample(resid0, replace=T)

      # New y-values under model = fitted + resampled residuals
```

```
      Yb <- y0 + residb
      matb[,j] <- Yb

      # Refit spline on bootstrap sample
      stat.outb <- my.dat.plot5(matb, i, j, zxlab, zylab, zmain,
                                zcol, do.sqrt, do.plot=F, in.boot=T)
      Ybp <- predict(stat.outb$smstrmod, matb[,i])$y

      # Store bootstrap fits depending on choice:
      # prediction vs confidence
      #pivotal vs percentile
      if(pred.bound){
        if(pivotal){
          bootmat <- rbind(bootmat, stat.outb$resid.mod + Ybp - y0)
        } else {
          bootmat <- rbind(bootmat, stat.outb$resid.mod + Ybp)
        }
      } else {
        if(pivotal){
          bootmat <- rbind(bootmat, Ybp - y0)
        } else {
          bootmat <- rbind(bootmat, Ybp)
        }
      }  # end if(pred.bound)...
    }    # end for loop
    alpha<-(1-conf.lev)/2
    my.quant<-function(x,a=alpha){quantile(x,c(a,1-a))}
    bounds<-apply(bootmat,2,my.quant)
    if(pivotal){
      bounds[1,]<-y0-bounds[1,]
      bounds[2,]<-y0-bounds[2,]
    }
    x<-mat[,i]
    if(do.sqrt){
      x<-sqrt(x)
    }
    o1<-order(x)
    lines(x[o1],bounds[1,o1],col=zcol+2)
    lines(x[o1],bounds[2,o1],col=zcol+2)
}

gui.bootstrap2 <-
function(){
library(tcltk)
#,pred.bound=T,conf.lev=.95,pivotal=T
inputs <- function(){

   x <- tclVar("NOAA")
   y <- tclVar("1")
   z <- tclVar("2")
   w<-tclVar("\"delta temp\"")
wa<-tclVar("\"disasters\"")
wb<-tclVar("\"Disasters vs warming\"")
zc<-tclVar("2")
qc<-tclVar("F")
rc<-tclVar("10000")
za<-tclVar("T")
zb<-tclVar(".95")
wc<-tclVar("T")

   tt <- tktoplevel()
   tkwm.title(tt,"Choose parameters for new function                  ")
   x.entry <- tkentry(tt, textvariable=x)
   y.entry <- tkentry(tt, textvariable=y)
   z.entry <- tkentry(tt, textvariable=z)
   w.entry<-tkentry(tt, textvariable=w)
wa.entry<-tkentry(tt,textvariable=wa)
wb.entry<-tkentry(tt,textvariable=wb)
zc.entry<-tkentry(tt,textvariable=zc)
qc.entry<-tkentry(tt,textvariable=qc)
rc.entry<-tkentry(tt,textvariable=rc)
za.entry<-tkentry(tt,textvariable=za)
zb.entry<-tkentry(tt,textvariable=zb)
wc.entry<-tkentry(tt,textvariable=wc)

   reset <- function()
    {
     tclvalue(x)<-""
     tclvalue(y)<-""
     tclvalue(z)<-""
        tclvalue(w)<-""
tclvalue(wa.entry)<-""
tclvalue(wb.entry)<-""
tclvalue(zc.entry)<-""
```

```
tclvalue(qc.entry)<-""
tclvalue(rc.entry)<-""
tclvalue(za.entry)<-""
tclvalue(zb.entry)<-""
tclvalue(wc.entry)<-""

       }

   reset.but <- tkbutton(tt, text="Reset", command=reset)

   submit <- function() {
     x <- tclvalue(x)
     y <- tclvalue(y)
     z <- tclvalue(z)
       w<-tclvalue(w)
        wa<-tclvalue(wa)
        wb<-tclvalue(wb)
        zc<-tclvalue(zc)
        qc<-tclvalue(qc)
        rc<-tclvalue(rc)
        za<-tclvalue(za)
zb<-tclvalue(zb)
wc<-tclvalue(wc)

     e <- parent.env(environment())
     e$x <- x
     e$y <- y
     e$z <- z
e$w<-w
e$wa<-wa
e$wb<-wb
e$zc<-zc
e$qc<-qc
e$rc<-rc
e$za<-za
e$zb<-zb
e$wc<-wc

        tkdestroy(tt)
   }

   submit.but <- tkbutton(tt, text="start", command=submit)
   tkgrid(tklabel(tt,text="Input data matrix"),columnspan=2)
   tkgrid(tklabel(tt,text="data"), x.entry, pady=10, padx =30)


   tkgrid(tklabel(tt,text="Input X column"),columnspan=2)
   tkgrid(tklabel(tt,text="i"), y.entry, pady=10, padx =30)


   tkgrid(tklabel(tt,text="Input Y column"),columnspan=2)
   tkgrid(tklabel(tt,text="j"), z.entry, pady=10, padx =30)

  tkgrid(tklabel(tt,text="Input xaxis label"),columnspan=2)
   tkgrid(tklabel(tt,text="Xaxis"), w.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input yaxis label"),columnspan=2)
   tkgrid(tklabel(tt,text="Yaxis"), wa.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input plot title"),columnspan=2)
   tkgrid(tklabel(tt,text="Plot title"), wb.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input color code"),columnspan=2)
   tkgrid(tklabel(tt,text="Color code"),zc.entry, pady=10, padx =30)

tkgrid(tklabel(tt,text="Input sqrt indicator"),columnspan=2)
   tkgrid(tklabel(tt,text="Sqrt=T"),qc.entry, pady=10, padx =30)

tkgrid(tklabel(tt,text="Input nboot"),columnspan=2)
   tkgrid(tklabel(tt,text="nboot"),rc.entry, pady=10, padx =30)

tkgrid(tklabel(tt,text="Prediction Bound?"),columnspan=2)
   tkgrid(tklabel(tt,text="Pbound=T"),za.entry, pady=10, padx =30)
tkgrid(tklabel(tt,text="Conf.level"),columnspan=2)
   tkgrid(tklabel(tt,text="Confidence"),zb.entry, pady=10, padx =30)
tkgrid(tklabel(tt,text="Pivotal=T"),columnspan=2)
   tkgrid(tklabel(tt,text="Pivotal"),wc.entry, pady=10, padx =30)


   tkgrid(submit.but, reset.but)

  tkwait.window(tt)
  return(c(x,y,z,w,wa,wb,zc,qc,rc,za,zb,wc))
}
```

```
#Now run the function like:
predictor_para <- inputs()
print(predictor_para)
mat<-eval(parse(text=predictor_para[1]))
ind1<-eval(parse(text=predictor_para[2]))
ind2<-eval(parse(text=predictor_para[3]))
xlab<-eval(parse(text=predictor_para[4]))
ylab<-eval(parse(text=predictor_para[5]))
maintitle<-eval(parse(text=predictor_para[6]))
zcol<-eval(parse(text=predictor_para[7]))
zsqrt<-eval(parse(text=predictor_para[8]))
znboot<-eval(parse(text=predictor_para[9]))
zpred<-eval(parse(text=predictor_para[10]))
zconf<-eval(parse(text=predictor_para[11]))
zpivot<-eval(parse(text=predictor_para[12]))

my.bootstrap2(mat,ind1,ind2,xlab,ylab, maintitle,zcol,zsqrt,znboot,zpred,zconf,zpivot)

}

# my.bootstrap3: XY bootstrap (resample rows of (x,y) pairs)
# des not assume residuals ~ model
# - Perturbs x-distribution too, so more “distribution-free”
# -same options for prediction vs confidence, pivotal vs percentile
my.bootstrap3 <-
  function(mat=NOAA, i=3, j=2, zxlab, zylab, zmain, zcol,
           do.sqrt=F, nboot=10000,
           pred.bound=T, conf.lev=.95, pivotal=T){

    par(mfrow=c(1,1))

    # Baseline fit (like in residual bootstrap)
    stat.out0 <- my.dat.plot5(mat, i, j, zxlab, zylab, zmain,
                              zcol, do.sqrt, do.plot=T, in.boot=F)

    bootmat <- NULL
    y0 <- predict(stat.out0$smstrmod, mat[,i])$y
    matb <- mat
    nm <- length(matb[,1])  # sample size

    for(i1 in 1:nboot){
      if(floor(i1/500) == (i1/500)){ print(i1) }

      # resample entire rows (X,Y pairs) with replacement
      zed <- sample(nm, replace=T)
      matb <- mat[zed,]

      # Fit spline on resampled (X,Y)
      stat.outb <- my.dat.plot5a(matb, mat, i, j, zxlab, zylab,
                                 zmain, zcol, do.sqrt, do.plot=F, in.boot=T)
      Ybp <- predict(stat.outb$smstrmod, mat[,i])$y

      #Store bootstrap fits depending on prediction/CI & pivotal/percentile
      if(pred.bound){
        if(pivotal){
          bootmat <- rbind(bootmat, stat.outb$resid.mod + Ybp - y0)
        } else {
          bootmat <- rbind(bootmat, stat.outb$resid.mod + Ybp)
        }
      } else {
        if(pivotal){
          bootmat <- rbind(bootmat, Ybp - y0)
        } else {
          bootmat <- rbind(bootmat, Ybp)
        }
      }
    }
    alpha<-(1-conf.lev)/2
    my.quant<-function(x,a=alpha){quantile(x,c(a,1-a))}
    bounds<-apply(bootmat,2,my.quant)
    if(pivotal){
      bounds[1,]<-y0-bounds[1,]
      bounds[2,]<-y0-bounds[2,]
    }
    x<-mat[,i]
    if(do.sqrt){
      x<-sqrt(x)
    }
    o1<-order(x)
    lines(x[o1],bounds[1,o1],col=zcol+2)
    lines(x[o1],bounds[2,o1],col=zcol+2)
  }

gui.bootstrapxy <-
```

```
function(){
library(tcltk)
#,pred.bound=T,conf.lev=.95,pivotal=T
inputs <- function(){

   x <- tclVar("NOAA")
   y <- tclVar("1")
   z <- tclVar("2")
   w<-tclVar("\"delta temp\"")
wa<-tclVar("\"disasters\"")
wb<-tclVar("\"Disasters vs warming\"")
zc<-tclVar("2")
qc<-tclVar("F")
rc<-tclVar("10000")
za<-tclVar("T")
zb<-tclVar(".95")
wc<-tclVar("T")

   tt <- tktoplevel()
   tkwm.title(tt,"Choose parameters for new function                    ")
   x.entry <- tkentry(tt, textvariable=x)
   y.entry <- tkentry(tt, textvariable=y)
   z.entry <- tkentry(tt, textvariable=z)
   w.entry<-tkentry(tt, textvariable=w)
wa.entry<-tkentry(tt,textvariable=wa)
wb.entry<-tkentry(tt,textvariable=wb)
zc.entry<-tkentry(tt,textvariable=zc)
qc.entry<-tkentry(tt,textvariable=qc)
rc.entry<-tkentry(tt,textvariable=rc)
za.entry<-tkentry(tt,textvariable=za)
zb.entry<-tkentry(tt,textvariable=zb)
wc.entry<-tkentry(tt,textvariable=wc)

   reset <- function()
    {
     tclvalue(x)<-""
     tclvalue(y)<-""
     tclvalue(z)<-""
        tclvalue(w)<-""
tclvalue(wa.entry)<-""
tclvalue(wb.entry)<-""
tclvalue(zc.entry)<-""
tclvalue(qc.entry)<-""
tclvalue(rc.entry)<-""
tclvalue(za.entry)<-""
tclvalue(zb.entry)<-""
tclvalue(wc.entry)<-""

       }

   reset.but <- tkbutton(tt, text="Reset", command=reset)

   submit <- function() {
     x <- tclvalue(x)
     y <- tclvalue(y)
     z <- tclvalue(z)
       w<-tclvalue(w)
        wa<-tclvalue(wa)
        wb<-tclvalue(wb)
        zc<-tclvalue(zc)
        qc<-tclvalue(qc)
        rc<-tclvalue(rc)
        za<-tclvalue(za)
zb<-tclvalue(zb)
wc<-tclvalue(wc)

     e <- parent.env(environment())
     e$x <- x
     e$y <- y
     e$z <- z
e$w<-w
e$wa<-wa
e$wb<-wb
e$zc<-zc
e$qc<-qc
e$rc<-rc
e$za<-za
e$zb<-zb
e$wc<-wc

        tkdestroy(tt)
   }

   submit.but <- tkbutton(tt, text="start", command=submit)
```

```
   tkgrid(tklabel(tt,text="Input data matrix"),columnspan=2)
   tkgrid(tklabel(tt,text="data"), x.entry, pady=10, padx =30)


   tkgrid(tklabel(tt,text="Input X column"),columnspan=2)
   tkgrid(tklabel(tt,text="i"), y.entry, pady=10, padx =30)


   tkgrid(tklabel(tt,text="Input Y column"),columnspan=2)
   tkgrid(tklabel(tt,text="j"), z.entry, pady=10, padx =30)

  tkgrid(tklabel(tt,text="Input xaxis label"),columnspan=2)
   tkgrid(tklabel(tt,text="Xaxis"), w.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input yaxis label"),columnspan=2)
   tkgrid(tklabel(tt,text="Yaxis"), wa.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input plot title"),columnspan=2)
   tkgrid(tklabel(tt,text="Plot title"), wb.entry, pady=10, padx =30)

 tkgrid(tklabel(tt,text="Input color code"),columnspan=2)
   tkgrid(tklabel(tt,text="Color code"),zc.entry, pady=10, padx =30)

tkgrid(tklabel(tt,text="Input sqrt indicator"),columnspan=2)
   tkgrid(tklabel(tt,text="Sqrt=T"),qc.entry, pady=10, padx =30)

tkgrid(tklabel(tt,text="Input nboot"),columnspan=2)
   tkgrid(tklabel(tt,text="nboot"),rc.entry, pady=10, padx =30)

tkgrid(tklabel(tt,text="Prediction Bound?"),columnspan=2)
   tkgrid(tklabel(tt,text="Pbound=T"),za.entry, pady=10, padx =30)
tkgrid(tklabel(tt,text="Conf.level"),columnspan=2)
   tkgrid(tklabel(tt,text="Confidence"),zb.entry, pady=10, padx =30)
tkgrid(tklabel(tt,text="Pivotal=T"),columnspan=2)
   tkgrid(tklabel(tt,text="Pivotal"),wc.entry, pady=10, padx =30)


   tkgrid(submit.but, reset.but)

  tkwait.window(tt)
  return(c(x,y,z,w,wa,wb,zc,qc,rc,za,zb,wc))
}
#Now run the function like:
predictor_para <- inputs()
print(predictor_para)
mat<-eval(parse(text=predictor_para[1]))
ind1<-eval(parse(text=predictor_para[2]))
ind2<-eval(parse(text=predictor_para[3]))
xlab<-eval(parse(text=predictor_para[4]))
ylab<-eval(parse(text=predictor_para[5]))
maintitle<-eval(parse(text=predictor_para[6]))
zcol<-eval(parse(text=predictor_para[7]))
zsqrt<-eval(parse(text=predictor_para[8]))
znboot<-eval(parse(text=predictor_para[9]))
zpred<-eval(parse(text=predictor_para[10]))
zconf<-eval(parse(text=predictor_para[11]))
zpivot<-eval(parse(text=predictor_para[12]))

my.bootstrap3(mat,ind1,ind2,xlab,ylab, maintitle,zcol,zsqrt,znboot,zpred,zconf,zpivot)

}

my.dat.plot5 <-
  function(mat=NOAA,i=3,j=2,zxlab,zylab,zmain,zcol,
           do.sqrt=F, do.plot=T, in.boot=T){
    # If in bootstrap mode, skip sqrt transform
    if(in.boot){ do.sqrt <- F }

    if(!do.sqrt){
      smstr     <- smooth.spline(mat[,i], mat[,j])         # flexible fit
      smstr.lin <- smooth.spline(mat[,i], mat[,j], df=2)   # linear fit
      if(do.plot){
        plot(mat[,i], mat[,j], xlab=zxlab, ylab=zylab, main=zmain)
        lines(smstr,     col=zcol)
        lines(smstr.lin, col=zcol+1)
      }
      # Save residuals for later (needed in bootstrap resampling)
      resid1 <- mat[,j] - predict(smstr,     mat[,i])$y
      resid2 <- mat[,j] - predict(smstr.lin, mat[,i])$y
    } else {
      # Same logic but apply sqrt transform to y before fitting
      smstr     <- smooth.spline(mat[,i], sqrt(mat[,j]))
      smstr.lin <- smooth.spline(mat[,i], sqrt(mat[,j]), df=2)
      if(do.plot){
```

```
        plot(mat[,i], sqrt(mat[,j]), xlab=zxlab, ylab=zylab, main=zmain)
        lines(smstr,     col=zcol)
        lines(smstr.lin, col=zcol+1)
      }
      resid1 <- sqrt(mat[,j]) - predict(smstr,     mat[,i])$y
      resid2 <- sqrt(mat[,j]) - predict(smstr.lin, mat[,i])$y
    }
    dfmod<-smstr$df
    dflin<-2
    ssmod<-sum(resid1^2)
    sslin<-sum(resid2^2)
    numss<-sslin-ssmod
    n1<-length(mat[,j])
    Fstat<-(numss/(dfmod-dflin))/(ssmod/(n1-dfmod))
    pvalue<-1-pf(Fstat,dfmod-dflin,n1-dfmod)
    stat.out<-
list(smstrmod=smstr,smstrlin=smstr.lin,resid.mod=resid1,resid.lin=resid2,dfmod=dfmod,dflin=dflin,F=Fstat,P=pvalue,n=n1)
  }

my.dat.plot5a <-
  function(mat=NOAA, mat0, i=3, j=2, zxlab, zylab, zmain, zcol,
           do.sqrt=F, do.plot=T, in.boot=T){
    # Variant of my.dat.plot5:
    # - Allows predictions on a second dataset mat0
    # - Used for bootstrap comparisons
    if(in.boot){ do.sqrt <- F }

    if(!do.sqrt){
      smstr <- smooth.spline(mat[,i], mat[,j])   # flexible fit
      # Predict spline values at new x's from mat0
      pstr  <- predict(smstr, mat0[,i])
      smstr$x <- pstr$x
      smstr$y <- pstr$y
      smstr.lin <- smooth.spline(mat[,i], mat[,j], df=2)
      if(do.plot){
        plot(mat[,i], mat[,j], xlab=zxlab, ylab=zylab, main=zmain)
        lines(smstr,     col=zcol)
        lines(smstr.lin, col=zcol+1)
      }
      resid1 <- mat[,j] - predict(smstr,     mat[,i])$y
      resid2 <- mat[,j] - predict(smstr.lin, mat[,i])$y
    } else {
      # sqrt-transformed version (similar adjustments as above)
      smstr <- smooth.spline(mat[,i], sqrt(mat[,j]))
      pstr  <- predict(smstr, mat0[,i])
      smstr$x <- pstr$x
      smstr$y <- pstr$y
      smstr.lin <- smooth.spline(mat[,i], sqrt(mat[,j]), df=2)
      if(do.plot){
        plot(mat[,i], sqrt(mat[,j]), xlab=zxlab, ylab=zylab, main=zmain)
        lines(smstr,     col=zcol)
        lines(smstr.lin, col=zcol+1)
      }
      resid1 <- sqrt(mat[,j]) - predict(smstr,     mat[,i])$y
      resid2 <- sqrt(mat[,j]) - predict(smstr.lin, mat[,i])$y
    }
    dfmod<-smstr$df
    dflin<-2
    ssmod<-sum(resid1^2)
    sslin<-sum(resid2^2)
    numss<-sslin-ssmod
    n1<-length(mat[,j])
    Fstat<-(numss/(dfmod-dflin))/(ssmod/(n1-dfmod))
    pvalue<-pf(Fstat,dfmod-dflin,n1-dfmod)
    stat.out<-
list(smstrmod=smstr,smstrlin=smstr.lin,resid.mod=resid1,resid.lin=resid2,dfmod=dfmod,dflin=dflin,F=Fstat,P=pvalue,n=n1)
  }
```